

POSTECH



TECH CHALLENGE

Tech Challenge é o projeto da fase que englobará os conhecimentos obtidos em todas as disciplinas da fase. Esta é uma atividade que, a princípio, deve ser desenvolvida em grupo. É importante atentar-se ao prazo de entrega, pois trata-se de uma atividade obrigatória, uma vez que vale 90% da nota de todas as disciplinas da fase.

Tema Central: Deploy de Modelo em Produção com Pipeline CI/CD, Monitoramento e Otimização de Latência.

Contexto

Um hospital de referência precisa de um sistema de triagem automática de exames de texto (laudos médicos) para classificar urgência (ex.: normal / atenção / urgente). O modelo central será um classificador de texto (NLP) leve, servido via API REST em container Docker. O foco do projeto é garantir que o ciclo de vida do modelo funcione com um pipeline CI/CD (GitHub Actions), orquestração básica de retreino (Airflow), monitoramento essencial (Prometheus + Grafana) e otimizações de latência.

Requisitos Obrigatórios

Repositório GitHub

- Pipeline CI/CD básico com GitHub Actions (ex.: lint → test → build).
- Script ou DAG Airflow simples para pipeline de treino/retreino.
- Dockerfile funcional para o serviço de inferência.
- Stack de monitoramento local: API + Prometheus + Grafana via Docker Compose.
- Histórico de commits semântico e organizado.

Vídeo (5 minutos — método STAR)

- **Situation:** Problema clínico e importância da triagem rápida.

- **Task:** Requisitos técnicos da fase (latência, CI/CD, monitoramento).
- **Action:** Arquitetura escolhida, como o modelo foi otimizado e como a monitoração foi configurada.
- **Result:** Demonstração do pipeline funcionando, latência alcançada e lições aprendidas.

Bibliotecas Requeridas

- **Scikit-Learn** ou framework de preferência — para o modelo base de classificação de texto (ex.: TF-IDF + Random Forest ou modelo leve similar).
- **FastAPI** — para construção da API.
- **Prometheus-client** — para instrumentação de métricas.
- **Airflow** — para orquestração de tarefas.

Boas Práticas Obrigatórias

- CI/CD contendo pelo menos 2 automações (ex.: verificação de código e testes).
- DAG Airflow funcional (ex.: carregamento de dados → treino → salvamento do modelo).
- Dashboard Grafana com pelo menos 3 painéis (ex.: total de requisições, latência/tempo de resposta, taxa de erro).
- Otimização de performance: Aplicação de pelo menos uma técnica vista em aula (ex.: conversão para ONNX, quantização básica ou pruning).

Etapas de Desenvolvimento (4 Etapas)

Etapa 1 — Decisão Arquitetural e API Inicial (Disciplina: Deploy em Nuvem)

Foco: Decisão de arquitetura de deploy e setup da aplicação.

Tarefas:

- Analisar qual estratégia de deploy em nuvem (ex.: AWS, Azure ou GCP) seria ideal para este cenário (batch vs. real-time) e documentar de forma textual no README.
- Criar uma API simples com FastAPI que receba o texto do laudo e retorne a classificação.
- Empacotar a API em um container Docker e medir o tempo de resposta (baseline de latência local).
- **Entregável:** API funcional rodando em Docker + documento/texto de decisão arquitetural no README.

Etapa 2 — CI/CD e Pipeline Automatizado (Disciplinas: CI/CD e Pipeline de Treino)

Foco: Automação básica do código e do modelo.

Tarefas:

- Criar um workflow no GitHub Actions que execute automaticamente testes simples (ex.: pytest) e o lint do código quando houver um push.
- Desenvolver uma DAG no Airflow para simular o processo de treinamento (ex.: uma task para ler um CSV de dados e uma task para treinar e salvar o modelo).
- **Entregável:** Workflow YAML no GitHub repositório + arquivo .py da DAG do Airflow.

Etapa 3 — Monitoramento e Observabilidade (Disciplinas: Monitoração de Performance e Serviços)

Foco: Stack de observabilidade para a API.

Tarefas:

- Instrumentar a API (usando prometheus_client) para expor métricas básicas: tempo de requisição e contagem de chamadas.

- Configurar um docker-compose.yml que suba a API, o Prometheus e o Grafana juntos.
- Criar um dashboard simples no Grafana para visualizar essas métricas.
- **Entregável:** Docker Compose rodando a stack completa + print/JSON do dashboard configurado.

Etapa 4 — Otimização de Latência e Entrega (Disciplina: Latência em Modelos Não Estruturados)

Foco: Modelo otimizado para inferência e consolidação.

Tarefas:

- Treinar o classificador de texto.
- Aplicar **uma** técnica de otimização de latência vista no curso (ex.: exportar o modelo treinado para o formato ONNX Runtime para inferência mais rápida, ou aplicar quantização).
- Comparar a latência do modelo original versus o modelo otimizado.
- Gravar o vídeo STAR demonstrando o projeto.
- **Entregável:** Modelo otimizado, resultados comparativos de latência e link do vídeo.

Critérios de Avaliação

Critério	Peso	Descrição
Modelagem e Otimização	20%	Modelo funcional de NLP, conversão/otimização (ex.: ONNX) bem-sucedida e melhoria de latência demonstrada.
CI/CD (GitHub Actions)	15%	Workflow configurado e rodando testes básicos.
Orquestração (Airflow)	15%	DAG funcional realizando as etapas de ingestão e treino.

Monitoramento	20%	Compose funcional (API + Prometheus + Grafana) com dashboard exibindo as métricas propostas.
Documentação (README)	15%	Explicação da arquitetura em nuvem escolhida e instruções claras de execução.
Vídeo STAR	15%	Clareza na demonstração técnica e explicação do impacto (≤ 5 min).

Dataset Sugerido

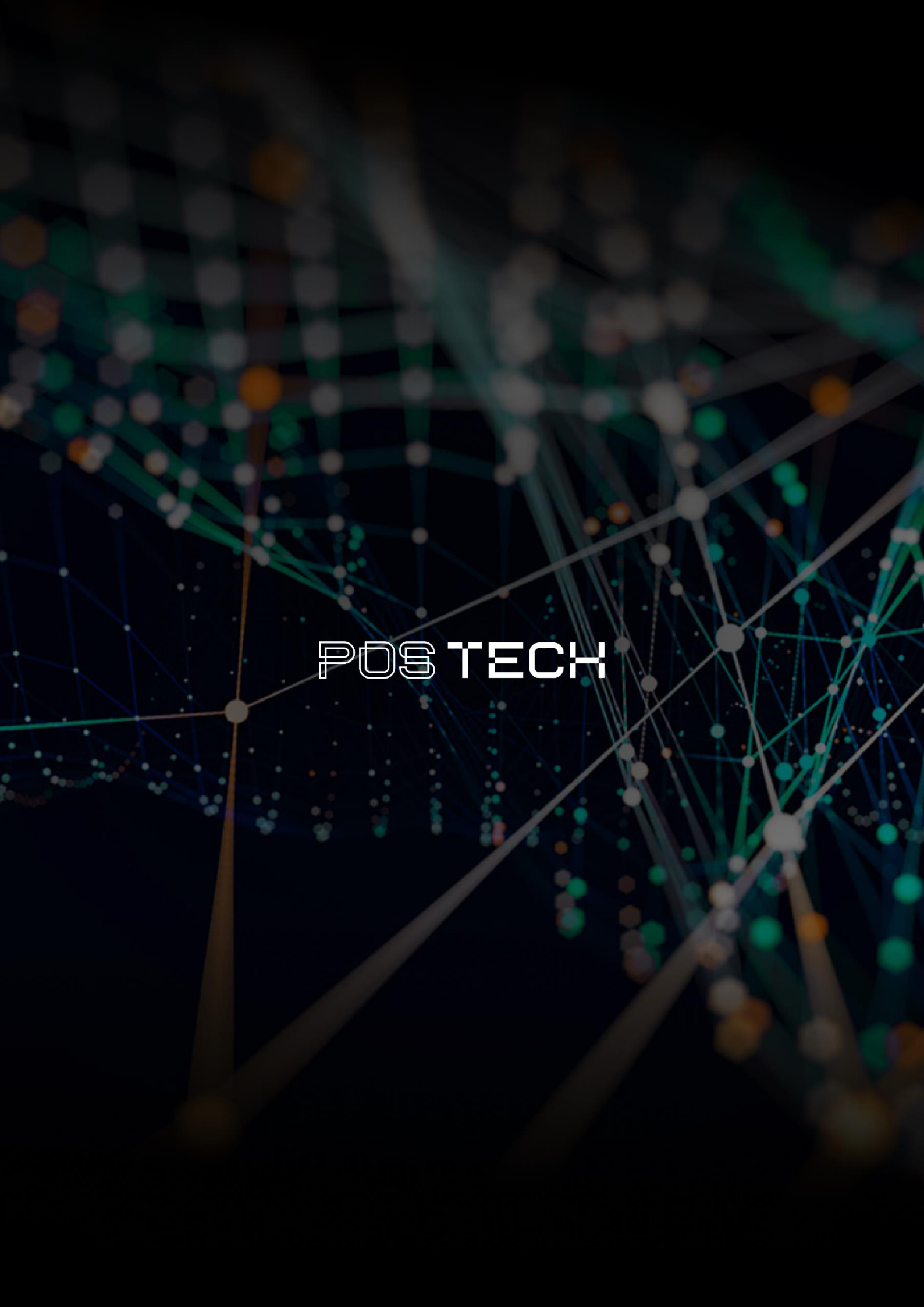
Recomendamos o uso de datasets públicos simples de classificação de textos médicos ou triagem.

- Exemplos: Medical Abstracts TC Corpus (Kaggle), recortes do MIMIC-III (open access) ou qualquer dataset tabular contendo uma coluna de texto (sintoma/laudo) e uma coluna de target (classificação/urgência) com pelo menos 2.000 amostras.

Passo a Passo Resumido

- **[Etapa 1]** Escolher arquitetura teórica + API FastAPI em Docker + Medir latência base.
- **[Etapa 2]** Configurar GitHub Actions (lint/test) + Criar DAG Airflow simples de treino.
- **[Etapa 3]** Configurar Docker Compose com Prometheus e Grafana + Gerar requisições para ver o gráfico.
- **[Etapa 4]** Aplicar otimização (ex.: converter modelo para ONNX) + Documentar resultados + Gravar o Vídeo.

Caso tenha qualquer dúvida, é só nos procurar no Discord!

The background is a dark, abstract network visualization. It features a complex web of thin, light-colored lines connecting numerous small, glowing nodes. The nodes are primarily white and light blue, with some larger, more prominent nodes in teal and orange. The overall effect is a sense of dynamic connectivity and data flow, typical of a digital or technological theme.

POSTECH